

Universidade Federal de Santa Catarina (UFSC)

INE5425-06208 – Modelagem e Simulação

Professor: Dr. Rafael Luiz Cancian

Mecanismo de Atualização Dinâmica do Parser no GenESyS

Desenvolvimento de Componente de Simulação (DCS)

Vicente Cardoso dos Santos (22102203)
Ricardo Henrique Medeiros Pereira (22100637)
Eduardo Bischoff Grasel (22200355)

30 de junho de 2026

Sumário

1	Introdução	1
2	Desenvolvimento	1
2.1	Visão geral: de contratos previstos a mecanismo funcional	1
2.2	Contrato de declaração de mudanças: ParserChangesInformation	1
2.2.1	Contrato previsto no branch 2026-1	1
2.2.2	Problema em aberto	2
2.2.3	Decisão arquitetural adotada	2
2.2.4	Implementação concreta	2
2.2.5	Evidência de validação	3
2.3	Contrato de extensão por elementos de modelo: __getParserChangesInformation()	3
2.3.1	Contrato previsto no branch 2026-1	3
2.3.2	Problema em aberto	4
2.3.3	Decisão arquitetural adotada	4
2.3.4	Implementação concreta	4
2.3.5	Evidência de validação	5
2.4	Contrato de geração: ParserManager::generateNewParser()	5
2.4.1	Contrato previsto no branch 2026-1	5
2.4.2	Problema em aberto	5
2.4.3	Decisão arquitetural adotada	5
2.4.4	Implementação concreta	5
2.4.5	Evidência de validação	7
2.5	Contrato de conexão: ParserManager::connectNewParser()	7
2.5.1	Contrato previsto no branch 2026-1	7
2.5.2	Problema em aberto	7
2.5.3	Decisão arquitetural adotada	7
2.5.4	Implementação concreta	8
2.5.5	Evidência de validação	9
2.6	Contrato de integração com o ciclo do modelo: ModelDataManager::insert() e Model::parseExpression()	9
2.6.1	Contrato previsto no branch 2026-1	9
2.6.2	Problema em aberto	9
2.6.3	Decisão arquitetural adotada	9
2.6.4	Implementação concreta	10
2.6.5	Evidência de validação	10
2.7	Contrato de substituição segura: Parser_if, ParserDefaultImpl2 e ownership do Sampler	11
2.7.1	Contrato previsto no branch 2026-1	11
2.7.2	Problema em aberto	11
2.7.3	Decisão arquitetural adotada	11
2.7.4	Implementação concreta	11
2.7.5	Evidência de validação	12
2.8	Pipeline completo implementado	12
2.9	Projeto de software e diagramas	13
2.9.1	Diagrama de classes	13
2.9.2	Diagrama de sequência	14

2.9.3	Diagrama de componentes	15
2.9.4	Diagrama de estados	15
3	Validação	16
3.1	Rastreabilidade entre requisitos, contratos e testes	16
3.2	Testes unitários e de integração	17
3.3	Comandos de validação	18
3.4	Experimento com plugin Demo	19
4	Conclusão	20

1 Introdução

O GenESyS é um simulador extensível por plugins. Essa característica permite que novos elementos de modelagem sejam incorporados ao simulador sem reescrever o núcleo da aplicação. Entretanto, a linguagem de expressões usada pelo simulador também precisa acompanhar essas extensões. Quando um plugin introduz novas funções, novos símbolos ou novas regras de acesso a elementos do modelo, o parser deve ser capaz de reconhecer essas construções.

O tema deste trabalho é o mecanismo de atualização dinâmica do parser do GenESyS. O objetivo é tornar funcional a infraestrutura já prevista no projeto para que plugins possam declarar alterações léxicas, sintáticas e semânticas, permitindo que o simulador gere, compile e carregue uma nova versão do parser em tempo de execução.

A solução implementada adota atualização automática sob demanda (*lazy reload*). Ao inserir no modelo uma extensão que fornece informações de parser, o modelo marca seu parser como desatualizado. A primeira chamada posterior a `Model::parseExpression()` dispara automaticamente a coleta das mudanças, a geração dos arquivos Flex/Bison, a compilação da biblioteca dinâmica e a substituição segura do parser ativo.

2 Desenvolvimento

2.1 Visão geral: de contratos previstos a mecanismo funcional

A comparação com `origin/2026-1` mostra que o GenESyS já possuía uma arquitetura preparada para um parser dinâmico, mas ainda não possuía o mecanismo completo. O branch base já continha classes, métodos virtuais, estruturas de retorno e marcadores nos arquivos Flex/Bison que indicavam onde uma extensão deveria declarar mudanças e onde o kernel deveria gerar e conectar um novo parser. Portanto, a decisão principal deste trabalho foi não criar uma rota paralela de interpretação de expressões. A solução foi concretizar os contratos que já estavam desenhados.

Essa decisão organizou a implementação em uma cadeia de responsabilidades. Primeiro, a extensão descreve o que precisa mudar na linguagem por meio de `ParserChangesInformation`. Depois, os elementos de modelo expõem essa descrição pelo hook `_getParserChangesInformation()`. Em seguida, o `ParserManager` agrega essas descrições, injeta os fragmentos nos arquivos-base do parser, invoca a toolchain Bison/Flex/C++ e produz uma biblioteca compartilhada. Por fim, o parser compilado é carregado pelo mecanismo de bibliotecas dinâmicas do sistema operacional, instanciado por uma factory C e conectado ao `Model` pela interface `Parser_if`.

O resultado é um mecanismo em que cada parte nova responde a uma lacuna específica do branch base: representação das mudanças, descoberta das extensões, geração de código, carregamento dinâmico, ativação sob demanda e preservação de ownership.

2.2 Contrato de declaração de mudanças: `ParserChangesInformation`

2.2.1 Contrato previsto no branch 2026-1

A classe `ParserChangesInformation` já existia em 2026-1 como o objeto de contrato entre uma extensão e o mecanismo de parser. O papel dela não era executar a mudança,

compilar arquivos ou carregar bibliotecas. Seu papel era carregar os fragmentos de texto que precisariam ser inseridos na linguagem do simulador. Isso é importante porque separa duas responsabilidades: a extensão declara o que precisa mudar; o kernel decide como aplicar essa mudança.

No contrato original, a classe já possuía seções relacionadas à gramática Bison. Nessa etapa sintática, um token é a unidade que a gramática consome depois que o texto da expressão foi analisado pelo lexer; um tipo semântico define que tipo de valor uma regra pode carregar; e uma produção descreve como tokens e subexpressões podem formar uma expressão válida. Por isso, campos como `includes`, `tokens`, `tipos`, `expressões`, `produções de expressão`, `atribuições` e `produções de função` indicavam que o projeto já previa um parser extensível por composição de fragmentos, e não por alteração manual direta do arquivo `bisonparser.yy` a cada novo plugin.

2.2.2 Problema em aberto

O problema era que o contrato ainda não representava todo o caminho necessário para uma extensão real de linguagem. Uma nova função no parser não exige apenas uma produção Bison; ela também exige que o analisador léxico reconheça o literal ou padrão textual daquela função. Sem campos dedicados ao lexer, a extensão ficaria parcialmente declarada: a gramática poderia saber o que fazer com um token, mas o Flex não saberia como produzir esse token.

Também havia um problema de semântica operacional. Um `ParserChangesInformation` vazio ainda era um objeto válido do ponto de vista de C++, mas não representava uma mudança. Sem um método central para responder se há conteúdo efetivo, cada consumidor teria que repetir comparações de campos vazios. Isso aumentaria a chance de inconsistência: um ponto poderia considerar apenas tokens e `includes`, enquanto outro poderia esquecer regras léxicas, por exemplo.

2.2.3 Decisão arquitetural adotada

A decisão foi manter `ParserChangesInformation` como objeto de transporte de dados, mas completar seu papel para o pipeline inteiro Bison/Flex. Bison cuida da estrutura sintática da expressão; Flex cuida do reconhecimento inicial do texto. Assim, a classe passou a representar tanto mudanças sintáticas quanto léxicas. A extensão continua sem saber onde ficam os marcadores nos arquivos-base; ela apenas preenche a seção correta do contrato.

Outra decisão foi concentrar a noção de “mudança efetiva” dentro da própria classe. O método `hasChanges()` tornou-se o critério único usado pelo restante do mecanismo para decidir se uma contribuição deve participar do reload. Com isso, o contrato deixa de ser apenas um agrupamento de strings e passa a oferecer também uma regra de validade operacional.

2.2.4 Implementação concreta

Para que uma extensão seja reconhecida desde a leitura do texto, o contrato precisa representar duas formas de alteração léxica. A primeira é uma regra léxica: um padrão que o lexer usa para reconhecer uma sequência de caracteres e produzir um token para a gramática. A segunda é um literal léxico: uma palavra, nome de função ou símbolo textual que passa a ser aceito diretamente pela linguagem.

Esses dois conceitos foram materializados nos campos `lexicalRules` e `lexicalLiterals`, com seus respectivos getters e setters. A escolha dos nomes segue a separação usada no arquivo Flex: regras mais gerais entram na região de regras léxicas, enquanto literais entram na região dedicada a textos reconhecidos explicitamente. Essas seções são usadas depois por `ParserManager::_injectFlexChanges()` para inserir conteúdo em `lexerparser.ll`.

Depois de ampliar o contrato, ainda era necessário responder uma pergunta operacional: este objeto realmente descreve alguma alteração? Para isso foi adicionado `hasChanges()`. O método percorre conceitualmente todas as seções do contrato, tanto as antigas quanto as novas, e retorna falso somente quando todas estão vazias.

```
bool ParserChangesInformation::hasChanges() const
{
    return !_includes.empty()
        || !_tokens.empty()
        || !_typeObjs.empty()
        || !_expressions.empty()
        || !_expressionProductions.empty()
        || !_assignments.empty()
        || !_functionProductions.empty()
        || !_lexicalRules.empty()
        || !_lexicalLiterals.empty();
}
```

O trecho mostra a mudança de significado da classe. Antes, o consumidor precisava interpretar manualmente se havia algo relevante. Agora, o próprio contrato responde isso. A inclusão das duas últimas condições também registra que uma mudança puramente léxica, mesmo sem produção Bison nova, já deve ser considerada alteração real do parser.

2.2.5 Evidência de validação

Os testes `ParserChangesInformationStartsEmpty`, `ParserChangesInformationStore`, `sAllConfiguredSections` e `ParserChangesInformationReportsWhetherAnySectionHasChanges` validam esse contrato. Eles comprovam que a classe inicia vazia, armazena todas as seções configuradas e detecta corretamente quando qualquer seção passa a conter uma mudança.

2.3 Contrato de extensão por elementos de modelo: `__getParserChangesInformation()`

2.3.1 Contrato previsto no branch 2026-1

`ModelDataDefinition` já oferecia o hook virtual protegido `__getParserChangesInformation()`. Esse ponto era arquiteturalmente adequado porque os plugins e extensões do GenESyS entram no modelo como definições de dados ou componentes derivados da infraestrutura do kernel. Logo, fazia sentido que o contrato de alteração de linguagem estivesse associado à própria classe de modelo que introduz o novo conceito.

O contrato previsto era: se uma classe de modelo precisa alterar a linguagem, ela sobrescreve esse método e retorna as alterações necessárias. Isso preserva o princípio de extensibilidade do GenESyS, pois o kernel não precisa conhecer tipos específicos. Ele precisa apenas chamar um método virtual comum.

2.3.2 Problema em aberto

Em 2026-1, a implementação padrão retornava um objeto novo e vazio. Isso cumpria o tipo de retorno, mas deixava a semântica ambígua. O consumidor não conseguia distinguir imediatamente uma classe que não participa do parser dinâmico de uma classe que participa, mas retornou um contrato sem conteúdo.

Essa ambiguidade também tinha consequência prática: quase todo elemento comum do modelo passaria a alocar um objeto vazio apenas para dizer, indiretamente, que não possui alteração. O mecanismo de agregação teria que deletar esses objetos, e qualquer esquecimento poderia virar vazamento ou regeneração indevida.

2.3.3 Decisão arquitetural adotada

A decisão foi separar explicitamente ausência de participação e participação vazia. Participar do contrato significa que uma classe de modelo está declarando ao kernel que deseja alterar a linguagem de expressões. Não participar significa que aquela classe deve ser ignorada pelo mecanismo de parser dinâmico.

Com essa distinção conceitual definida, `nullptr` passou a significar: esta classe não contribui com alterações de parser. Um objeto `ParserChangesInformation` não nulo passa a significar: esta classe está se apresentando ao mecanismo de parser dinâmico. Ainda assim, esse objeto só deve ser aceito se `hasChanges()` indicar conteúdo real.

Com isso, o contrato fica em dois estágios. O ponteiro responde se existe participação. O método `hasChanges()` responde se a participação possui dados aplicáveis. Essa divisão também torna o código consumidor mais legível.

2.3.4 Implementação concreta

A classe base precisa representar o caso comum: a maioria das definições de dados do simulador não altera a linguagem. Por isso, a implementação padrão de `ModelDataDefinition` passou a retornar `nullptr`. Esse retorno não é uma falha; ele é o sinal escolhido para “não participo do contrato de parser dinâmico”. O trecho explicita essa regra:

```
ParserChangesInformation* ModelDataDefinition::_getParserChangesInformation() {  
    return nullptr; // if there is no override, return no changes  
}
```

No ponto consumidor, era necessário transformar essa convenção em uma decisão de estado do modelo. O gerenciador de dados é quem sabe que uma definição acabou de entrar no modelo; portanto, ele passou a consultar o hook logo após a inserção. A marcação do parser como potencialmente desatualizado ocorre somente quando existe um objeto de mudança e esse objeto contém alterações reais:

```
ParserChangesInformation* changes = anElement->_getParserChangesInformation();  
if (changes != nullptr && changes->hasChanges()) {  
    _parentModel->setParserIsStale(true);  
}  
delete changes;
```

Esse trecho resolve três pontos ao mesmo tempo. Primeiro, classes sem override não participam do parser. Segundo, objetos vazios não disparam recompilação. Terceiro, a responsabilidade de liberar o objeto retornado fica clara no ponto consumidor.

2.3.5 Evidência de validação

`DynamicParserExtensionAddsNewFunction` demonstra uma classe derivada que sobreescreve o hook e declara uma nova função. `AggregateChangesCombinesMultipleParserExtensions` demonstra que mais de uma extensão pode preencher o contrato ao mesmo tempo, sem que o kernel precise conhecer tipos concretos.

2.4 Contrato de geração: `ParserManager::generateNewParser()`

2.4.1 Contrato previsto no branch 2026-1

`ParserManager` já possuía a assinatura `generateNewParser(ParserChangesInformation*)`. Também já existiam duas estruturas de dados importantes: `NewParser`, para registrar os caminhos dos artefatos gerados, e `GenerateNewParserResult`, para retornar sucesso, mensagens de Bison, mensagens de Flex, mensagens de compilação e o novo parser produzido.

Isso mostra que o contrato de geração já estava desenhado: o kernel deveria receber mudanças declaradas e devolver um pacote de artefatos de parser. O que ainda não existia era o conteúdo operacional desse contrato.

2.4.2 Problema em aberto

O problema central era transformar strings de contrato em um parser real. Isso envolve etapas heterogêneas: manipulação de arquivos, conhecimento dos marcadores Bison/Flex, execução de ferramentas externas, compilação C++ e montagem de um retorno diagnóstico.

Também havia um risco de arquitetura: uma implementação simples poderia gerar uma gramática totalmente nova, duplicando o parser do GenESyS e criando duas fontes de verdade. Isso seria frágil, porque qualquer mudança no parser original teria que ser replicada no gerador dinâmico.

2.4.3 Decisão arquitetural adotada

A decisão foi tratar os arquivos-base do parser como fonte de verdade. O gerador não reconstrói a linguagem; ele injeta fragmentos nas regiões que já existiam para plugins. Essa escolha respeita os marcadores do branch 2026-1, reduz o tamanho da alteração e mantém o parser dinâmico alinhado ao parser estático.

Outra decisão foi gerar todos os artefatos em um diretório de trabalho. Dessa forma, os arquivos originais não são alterados durante a execução. O resultado gerado é uma biblioteca compartilhada separada, `genesys_parser_dynamic.so`, que pode ser carregada ou rejeitada sem corromper o parser ativo.

2.4.4 Implementação concreta

O método de geração precisa transformar uma descrição abstrata de mudanças em arquivos e, depois, em uma biblioteca carregável. Para isso, ele trabalha com quatro tipos de artefato. Os arquivos-base são a fonte da linguagem original do GenESyS. O diretório de trabalho é uma área descartável onde os arquivos modificados são gravados sem alterar a árvore original. Os arquivos de suporte são dependências mínimas para compilar o parser gerado. O resultado diagnóstico é o objeto que informa se cada etapa funcionou.

Com esses papéis definidos, o primeiro passo de `generateNewParser()` é validar se há mudanças. Se o ponteiro for nulo, o método retorna falha com mensagem de diagnóstico. Em seguida, cria o diretório de trabalho e localiza os arquivos-base `bisonparser.yy` e `lexerparser.ll`. Essa busca usa `_findBaseFile()`, porque o executável pode ser iniciado de diretórios diferentes.

Depois, o método lê o conteúdo dos arquivos-base e injeta as alterações. A injeção não é uma concatenação no fim do arquivo: ela ocorre entre marcadores preparados no parser original. Um marcador de início e um marcador de fim delimitam a região em que fragmentos de plugins podem entrar. A função genérica de injeção procura esses dois limites. Se o bloco estiver vazio, não faz nada; se os marcadores não forem encontrados, retorna erro:

```
bool ParserManager::_injectSection(std::string& content,
    const std::string& beginMarker,
    const std::string& endMarker,
    const std::string& block) {
    if (block.empty()) return true;
    size_t beginPos = content.find(beginMarker);
    if (beginPos == std::string::npos) return false;
    size_t endPos = content.find(endMarker, beginPos);
    if (endPos == std::string::npos) return false;
    content.insert(endPos, block + "\n");
    return true;
}
```

Cada seção do contrato precisa ir para uma região diferente do parser. Tokens e produções pertencem ao arquivo Bison; regras e literais de reconhecimento textual pertencem ao arquivo Flex. As funções `_injectBisonChanges()` e `_injectFlexChanges()` implementam esse mapeamento entre campos do contrato e marcadores concretos. Assim, `ParserChangesInformation` não precisa conhecer os nomes dos marcadores:

```
ok &= _injectSection(content,
    "/****begin_Tokens_plugins****/",
    "/****end_Tokens_plugins****/",
    changes->gettokens());

ok &= _injectSection(content,
    "%{/****begin_Lexical_plugins***%/}",
    "%{/****end_Lexical_plugins***%/}",
    changes->getlexicalRules());
```

Após a injeção, `generateNewParser()` escreve os arquivos modificados no diretório de trabalho. Essa etapa cria uma cópia derivada dos arquivos-base, preservando os originais. Em seguida, copia os arquivos de suporte necessários para compilar o parser fora da árvore original. O driver conecta o scanner e o parser, `obj_t` fornece tipos auxiliares usados pela gramática, e a factory cria a instância do parser dentro da biblioteca dinâmica. Esse passo é importante porque a biblioteca gerada precisa conter tudo que é necessário para ser carregada e instanciar um parser.

Por fim, o método invoca a toolchain. Bison transforma a gramática em código C++; Flex transforma as regras léxicas em scanner; o compilador C++ combina parser, scanner e arquivos de suporte em uma biblioteca compartilhada. Cada ferramenta recebe um campo próprio de mensagem no resultado. Assim, uma falha em Bison não fica misturada com uma falha de linkedição C++:

```

if (!_invokeBison(workDir, bisonOut, result.bisonMessages)) {
    result.compilationMessages = "Bison generation failed";
    return result;
}
if (!_invokeFlex(workDir, flexOut, result.lexMessages)) {
    result.compilationMessages = "Flex generation failed";
    return result;
}
if (!_compileDynamicLibrary(workDir, result.compilationMessages)) {
    return result;
}

result.result = true;
result.newParser.compiledParserFilename =
    workDir + "genesys_parser_dynamic.so";

```

2.4.5 Evidência de validação

`DynamicParserExtensionAddsNewFunction` valida o caminho completo para a função `parsertest`: declaração da extensão, geração, compilação, conexão e avaliação. `AggregateChangesCombinesMultipleParserExtensions` valida que o gerador recebe uma combinação de fragmentos de múltiplas extensões.

2.5 Contrato de conexão: `ParserManager::connectNewParser()`

2.5.1 Contrato previsto no branch 2026-1

`ParserManager` já declarava `connectNewParser(NewParser)`. Esse contrato complementa a geração: depois que `generateNewParser()` produz uma biblioteca, algum ponto do kernel precisa carregar essa biblioteca e transformar o artefato em um parser ativo do modelo.

O branch base indicava a intenção, mas não definia as regras de runtime: qual símbolo procurar, como tratar erro de carga, como substituir o parser antigo, como gerenciar a biblioteca anterior e como preservar objetos compartilhados entre parser antigo e novo.

2.5.2 Problema em aberto

Carregar o parser dinâmico não equivale a criar um objeto C++ local. A biblioteca compilada em runtime tem seus próprios símbolos, e nomes C++ podem sofrer *name mangling*. Se o gerenciador tentasse procurar diretamente um construtor C++, o contrato seria frágil e dependente de ABI.

Além disso, a troca precisa ser transacional do ponto de vista do modelo. Se a biblioteca não existir, se a carga dinâmica falhar, se a resolução do símbolo da factory não funcionar, ou se a própria factory lançar exceção, o parser antigo precisa continuar funcionando. A falha de atualização não pode deixar o modelo sem parser.

2.5.3 Decisão arquitetural adotada

A decisão foi criar uma fronteira C estável para instanciar o parser dinâmico. Uma fronteira C, neste contexto, é uma função exportada com nome previsível, sem decoração de nomes do C++. Isso permite que o carregador dinâmico encontre a função pelo texto exato

do seu nome. Essa função recebeu o nome `genesys_createParser` e foi declarada com `extern "C"`. O `ParserManager` não conhece o construtor concreto nem depende do nome C++ gerado pelo compilador; ele procura um símbolo simples e conhecido.

Também foi necessário escolher como a biblioteca seria carregada. Carregar a biblioteca significa pedir ao sistema operacional que abra o arquivo `.so` e prepare seus símbolos para uso. Resolver um símbolo significa procurar, dentro dessa biblioteca carregada, o endereço de uma função exportada. Para a primeira operação foi usado `dlopen`; para a segunda, `dlsym`. A chamada a `dlopen` usa `RTLD_NOW | RTLD_DEEPBIND`. `RTLD_NOW` antecipa erros de resolução de símbolos para o momento da carga. `RTLD_DEEPBIND` ajuda a biblioteca dinâmica a preferir seus próprios símbolos, reduzindo conflito com símbolos do parser já carregado no processo.

2.5.4 Implementação concreta

Antes de carregar qualquer código externo, o método precisa garantir que possui as condições mínimas para uma troca segura. Essas pré-condições são: existe um modelo associado, existe um caminho para a biblioteca compilada e esse arquivo realmente está no sistema de arquivos. `connectNewParser()` começa validando esses pontos. Cada erro adiciona uma mensagem ao ponteiro opcional `errorMessage`. Isso permite que `Model::parseExpression()` reporte ao usuário por que o lazy reload falhou.

Depois da validação, ocorre uma sequência em três etapas. Primeiro, o arquivo `.so` é aberto e o sistema devolve um handle, isto é, uma referência opaca para a biblioteca carregada. Segundo, o código procura dentro desse handle a função `factory`. Terceiro, essa função será chamada para criar o parser. O trecho abaixo mostra as duas primeiras etapas:

```
void* newLibraryHandle =
    dlopen(newParser.compiledParserFilename.c_str(),
           RTLD_NOW | RTLD_DEEPBIND);

auto* createFn =
    reinterpret_cast<Parser_if* (*)(Model*, Sampler_if*)>(&
        dlsym(newLibraryHandle, "genesys_createParser"));
```

Uma `factory` é uma função cuja responsabilidade é criar objetos sem expor ao chamador os detalhes do construtor concreto. Neste caso, a `factory` exportada pela biblioteca dinâmica recebe o modelo e o sampler, cria a implementação concreta e devolve um ponteiro pela interface `Parser_if`. Assim, o kernel continua dependendo da abstração, não da classe concreta:

```
extern "C" Parser_if* genesys_createParser(Model* model,
                                           Sampler_if* sampler) {
    return new ParserDefaultImpl2(model, sampler);
}
```

Antes de chamar a `factory`, existe uma preparação de estado compartilhado. O sampler é o objeto usado pelo parser para funções probabilísticas; portanto, ele deve sobreviver à troca. O método retira temporariamente o sampler do parser antigo para que o novo parser possa recebê-lo. Se a `factory` falhar, o sampler é devolvido ao parser antigo e a biblioteca nova é fechada. Esse é o ponto que torna a troca segura em falha:

```
Parser_if* oldParser = _model->getParser();
Sampler_if* sampler =
    oldParser != nullptr ? oldParser->releaseSampler() : nullptr;
```

```

try {
    newParserInstance = createFn(_model, sampler);
} catch (...) {
    if (oldParser != nullptr && sampler != nullptr) {
        oldParser->setSamplerOwned(sampler);
    }
    dlclose(newLibraryHandle);
    return false;
}

```

Quando a criação funciona, o novo handle passa a ser o handle ativo, `Model::setParser()` substitui a instância e o handle antigo é fechado. Assim, o parser anterior só é descartado depois que o novo parser já foi criado com sucesso.

2.5.5 Evidência de validação

`ConnectNewParserRejectsMissingLibraryWithMessage` valida a rejeição de uma biblioteca inexistente com mensagem de erro. `LazyDynamicParserFailurePreservesExistingParser` valida que uma falha no reload não inutiliza o parser anterior.

2.6 Contrato de integração com o ciclo do modelo: `ModelDataManager::insert()` e `Model::parseExpression()`

2.6.1 Contrato previsto no branch 2026-1

O GenESyS já possuía dois pontos naturais para integrar a atualização do parser ao ciclo de vida do modelo. `ModelDataManager::insert()` é o ponto por onde uma nova definição de dados entra no modelo. `Model::parseExpression()` é o ponto por onde a linguagem de expressões é efetivamente usada.

Esses pontos já pertenciam ao modelo, portanto a solução poderia ser integrada ao fluxo normal do simulador. Não seria necessário criar uma API externa obrigatória para “recarregar parser” sempre que um plugin fosse inserido.

2.6.2 Problema em aberto

Havia uma escolha de momento. Regenerar o parser dentro de `insert()` deixaria o modelo imediatamente atualizado, mas faria trabalho caro no momento de montagem do modelo. Pior: várias extensões poderiam ser inseridas em sequência, gerando recompilações repetidas antes de qualquer expressão ser avaliada.

Por outro lado, depender de um comando manual resolveria o custo, mas criaria uma interação frágil. O usuário poderia carregar uma extensão, escrever uma expressão nova e receber erro apenas porque esqueceu de pedir a regeneração do parser.

2.6.3 Decisão arquitetural adotada

A decisão foi usar lazy reload. Nesse desenho, “stale” significa que o parser ativo ainda funciona, mas pode estar desatualizado em relação às extensões já inseridas no modelo. A inserção apenas marca esse estado quando encontra uma extensão com mudanças reais. A primeira chamada posterior a `parseExpression()` executa a atualização. Esse desenho combina as duas necessidades: evita recompilações desnecessárias e preserva comportamento automático para quem usa a linguagem.

2.6.4 Implementação concreta

No lado da inserção, o objetivo é apenas registrar que a linguagem talvez precise ser atualizada. Por isso, `ModelDataManager::insert()` não chama Bison, Flex nem compilador. Ele materializa os dados internos do elemento, consulta o contrato de parser e, se houver mudança, marca o modelo como potencialmente desatualizado:

```
ModelDataDefinition::CreateInternalData(anElement);
ParserChangesInformation* changes =
    anElement->_getParserChangesInformation();
if (changes != nullptr && changes->hasChanges()) {
    _parentModel->setParserIsStale(true);
}
delete changes;
```

No lado da avaliação, o objetivo é sincronizar a linguagem exatamente antes de usá-la. `Model::parseExpression()` verifica a flag antes de delegar ao parser ativo. Quando a flag está marcada, o método limpa o estado stale para evitar reentradas repetidas, agrega as mudanças declaradas pelas extensões e combina as seções em um único objeto de mudanças:

```
if (_parserIsStale) {
    _parserIsStale = false;
    auto allChanges = _parserManager->aggregateChanges();
    auto* combined = new ParserChangesInformation();
    for (auto* ch : allChanges) {
        combined->setTokens(combined->gettokens() + ch->gettokens());
        combined->setLexicalRules(
            combined->getlexicalRules() + ch->getlexicalRules());
    }
    auto result = _parserManager->generateNewParser(combined);
    ...
}
```

O trecho acima é reduzido, mas mostra a ideia central: as contribuições são concatenadas por seção antes de gerar o parser. No código completo, o mesmo padrão é aplicado a includes, tipos, expressões, produções, atribuições, funções e literais léxicos.

Depois da geração, `parseExpression()` tenta conectar o parser. Se a conexão falhar, retorna erro sem avaliar a expressão nova. Se a geração falhar, monta uma mensagem com saídas de Bison, Flex e compilação. Se tudo funcionar, o método segue para `_parser->parse()`, agora com o parser atualizado.

2.6.5 Evidência de validação

`LazyDynamicParserReloadsOnNextParseExpression` valida exatamente esse fluxo: inserir extensão, observar o parser potencialmente desatualizado e disparar reload na chamada seguinte a `parseExpression()`. `LazyDynamicParserFailurePreservesExistingParser` valida que falhas nesse caminho não removem o parser anterior.

2.7 Contrato de substituição segura: `Parser_if`, `ParserDefaultImpl2` e ownership do `Sampler`

2.7.1 Contrato previsto no branch 2026-1

O modelo já acessava o parser por meio da interface `Parser_if`, e `ParserDefaultImpl2` já era a implementação concreta ligada ao driver Bison/Flex. Essa separação já era uma base adequada para substituição dinâmica, porque o restante do simulador não deveria depender da classe concreta do parser.

O parser também se relacionava com `Sampler_if`, usado por funções probabilísticas da linguagem. Portanto, trocar o parser em runtime não significa apenas substituir um objeto de parsing; significa preservar também um recurso usado pela semântica das expressões.

2.7.2 Problema em aberto

A troca dinâmica cria um problema clássico de ownership. Se o parser antigo destruir o sampler durante sua destruição, o novo parser perde o gerador que deveria continuar servindo às funções probabilísticas. Se o sampler for entregue ao novo parser, mas o antigo continuar acreditando que o possui, pode ocorrer dupla liberação. Se nenhum dos dois assumir ownership, ocorre vazamento.

Esse problema não podia ser resolvido apenas dentro de `ParserManager`, porque a posse efetiva estava encapsulada na implementação do parser. Era necessário ampliar a interface.

2.7.3 Decisão arquitetural adotada

A decisão foi introduzir operações explícitas de transferência na interface `Parser_if`. Ownership, aqui, significa a regra que define quem deve destruir o `Sampler_if`. A interface precisava expressar duas operações: receber um sampler assumindo responsabilidade por destruí-lo, e entregar o sampler atual sem destruí-lo. A primeira operação foi representada por `setSamplerOwned()`; a segunda, por `releaseSampler()`. Para manter compatibilidade, esses métodos têm implementação padrão na interface, mas `ParserDefaultImpl2` os sobrescreve porque conhece o wrapper interno e sua regra de destruição.

2.7.4 Implementação concreta

Para transformar a regra de ownership em estado concreto, a implementação precisa guardar se o sampler atualmente associado ao wrapper deve ou não ser destruído pelo parser. Esse estado foi representado pela flag `_ownsSampler` em `ParserDefaultImpl2`. Quando o sampler é externo, o parser apenas usa o objeto, mas não o destrói. Quando o sampler é transferido para o parser, ele passa a ser responsável pela vida útil desse objeto. O método `setSampler()` continua existindo para o caso externo; `setSamplerOwned()` cobre o caso de posse transferida:

```
void ParserDefaultImpl2::setSampler(Sampler_if* sampler) {
    _setSamplerInternal(sampler, false);
}

void ParserDefaultImpl2::setSamplerOwned(Sampler_if* sampler) {
    _setSamplerInternal(sampler, true);
}
```

Para mover o sampler do parser antigo para o novo, não basta ler o ponteiro. É preciso retirar também a responsabilidade de destruição do parser antigo. O método `releaseSampler()` é a peça central dessa transferência: ele captura o ponteiro atual, desliga o ownership e remove o sampler do wrapper para impedir que o parser antigo o destrua:

```
Sampler_if* ParserDefaultImpl2::releaseSampler() {
    Sampler_if* s = _wrapper.getSampler();
    _ownsSampler = false;
    _wrapper.setSampler(nullptr);
    return s;
}
```

No nível do modelo, a troca de parser precisa executar a transferência nessa ordem: retirar o sampler do parser antigo, entregar o sampler ao novo parser com posse definida, destruir o parser antigo e atualizar o ponteiro ativo. `Model::setParser()` usa o contrato da interface para realizar essa sequência:

```
Sampler_if* currentSampler =
    (_parser != nullptr) ? _parser->releaseSampler() : nullptr;
if (parser != nullptr && currentSampler != nullptr) {
    parser->setSamplerOwned(currentSampler);
}
delete _parser;
_parser = parser;
```

Com isso, a troca do parser fica alinhada à semântica de ownership: o sampler sai do parser antigo antes da destruição e entra no parser novo com posse definida.

2.7.5 Evidência de validação

Os testes `ParserDefaultImpl2SetSampler*` validam as situações de sampler externo, sampler possuído e liberação durante troca. A validação também aparece indiretamente nos testes dinâmicos, pois o parser novo consegue continuar avaliando expressões após a substituição.

2.8 Pipeline completo implementado

A partir desses contratos, o fluxo final implementado segue uma sequência determinística. Uma extensão derivada de `ModelDataDefinition` declara fragmentos de linguagem em `ParserChangesInformation`. Esses fragmentos podem descrever tanto como o texto será reconhecido pelo lexer quanto como a expressão será estruturada pela gramática. Ao ser inserida, `ModelDataManager::insert()` detecta a contribuição e marca o parser como potencialmente desatualizado. Quando `Model::parseExpression()` é chamado, o modelo pede ao `ParserManager` que agregue as mudanças, combine os fragmentos, gere os arquivos Flex/Bison modificados, compile a biblioteca compartilhada e conecte o novo parser. A expressão então é avaliada pelo parser atualizado.

Esse pipeline deixa clara a fronteira entre extensão, kernel e runtime. A extensão não compila nada; ela apenas declara. O kernel não conhece uma extensão específica; ele agrega contratos. O runtime não altera o modelo diretamente; ele carrega uma biblioteca que continua obedecendo à interface `Parser_if`.

Contrato previsto	Decisão arquitetural	Implementação concreta	Validação
<code>ParserChangesInformation</code>	Objeto de transporte para fragmentos sintáticos e léxicos, com detecção de mudança real.	Seções léxicas e <code>hasChanges()</code> .	<code>ParserChangesInformation*</code>
<code>_getParserChangesInformation()</code>	Participação no contrato passa a ser explícita: ausência, objeto vazio e mudança real deixam de ser confundidos.	Hook padrão alterado e extensões de teste sobrescrevendo o contrato.	<code>DynamicParserExtensionAddNewFunction</code>
<code>generateNewParser()</code>	Gerar parser por injeção nos marcadores existentes, sem reescrever a gramática inteira.	<code>_injectBisonChanges()</code> , <code>_injectFlexChanges()</code> , Bison, Flex e compilação da <code>.so</code> .	<code>AggregateChangesCombinesMultipleParserExtensions</code>
<code>connectNewParser()</code>	Carregar biblioteca por fronteira C estável e preservar parser antigo em falha.	Carregamento dinâmico, resolução de factory e criação por <code>genesys_createParser</code> .	<code>ConnectNewParserRejectsMissingLibraryWithMessage</code>
Ciclo do Model	Lazy reload: marcar parser potencialmente desatualizado na inserção e regenerar na primeira avaliação.	<code>ModelDataManager::insert()</code> e <code>Model::parseExpression()</code> .	<code>LazyDynamicParserReloadsOnNextParseExpression</code>
Ownership do sampler	Definir quem destrói o sampler durante a troca de parser.	<code>releaseSampler()</code> , <code>setSamplerOwned()</code> e <code>Model::setParser()</code> .	<code>ParserDefaultImpl2SetSampler*</code>

2.9 Projeto de software e diagramas

Os diagramas a seguir sintetizam a mesma linha de raciocínio usada nas subseções anteriores. Eles não documentam apenas quais classes existem; cada diagrama explicita uma decisão arquitetural: como contratos abstratos se conectam ao modelo, como o lazy reload acontece, como o pipeline de geração cruza fronteiras entre kernel e toolchain, e como o estado do parser permanece seguro em caso de falha.

2.9.1 Diagrama de classes

A Figura 1 mostra como os contratos foram materializados em classes. `ModelDataDefinition` representa o contrato de extensão, `ParserChangesInformation` representa o contrato de declaração de mudanças, `ParserManager` concretiza os contratos de geração e conexão, e `Parser_if` preserva a compatibilidade do restante do simulador com qualquer parser ativo. A presença de `ParserFactory` mostra a decisão de usar uma fronteira C para instanciar o parser gerado dinamicamente.

2.9.3 Diagrama de componentes

A Figura 3 separa as fronteiras de responsabilidade adotadas. A extensão declara, o kernel agrega e coordena, e a toolchain gera um artefato dinâmico carregado em runtime. Essa divisão é a decisão que impede que plugins precisem conhecer detalhes da gramática Bison, do reconhecimento Flex, da carga dinâmica por `dlopen` ou da regra de ownership do parser.

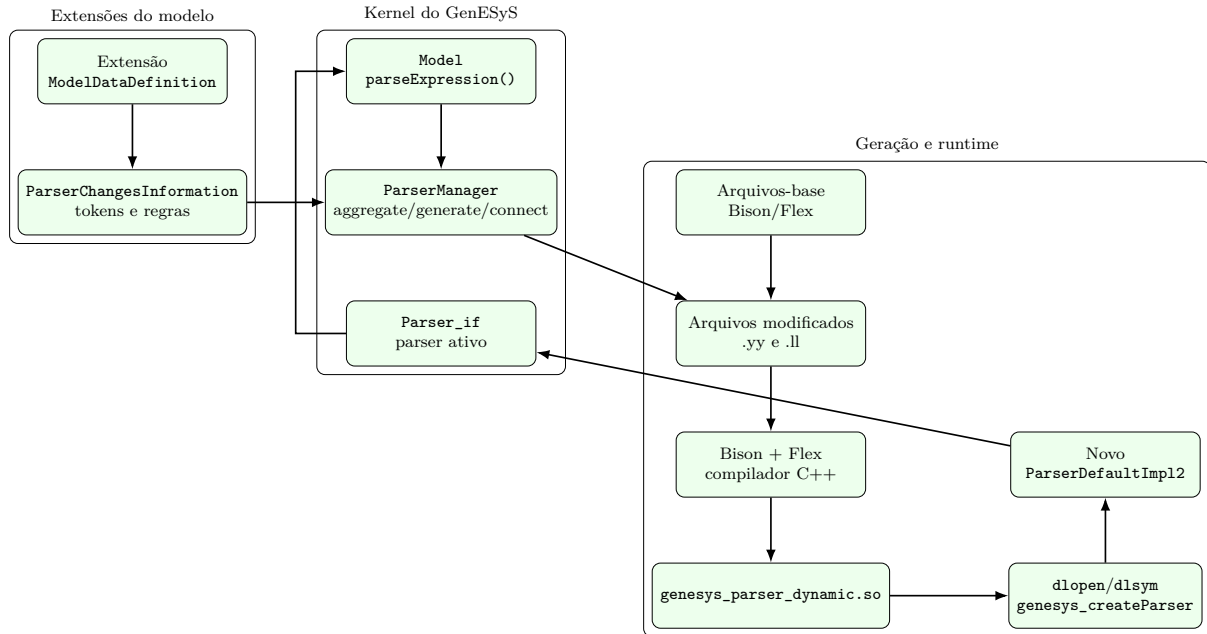


Figura 3: Componentes envolvidos na geração, compilação e carregamento do parser.

2.9.4 Diagrama de estados

A Figura 4 descreve o ciclo de vida resultante das decisões anteriores. O estado *stale* representa uma promessa de atualização ainda não executada; *generating* representa a execução concreta da toolchain; e *failed* explicita a regra de robustez: erro de geração ou conexão não substitui o parser antigo.

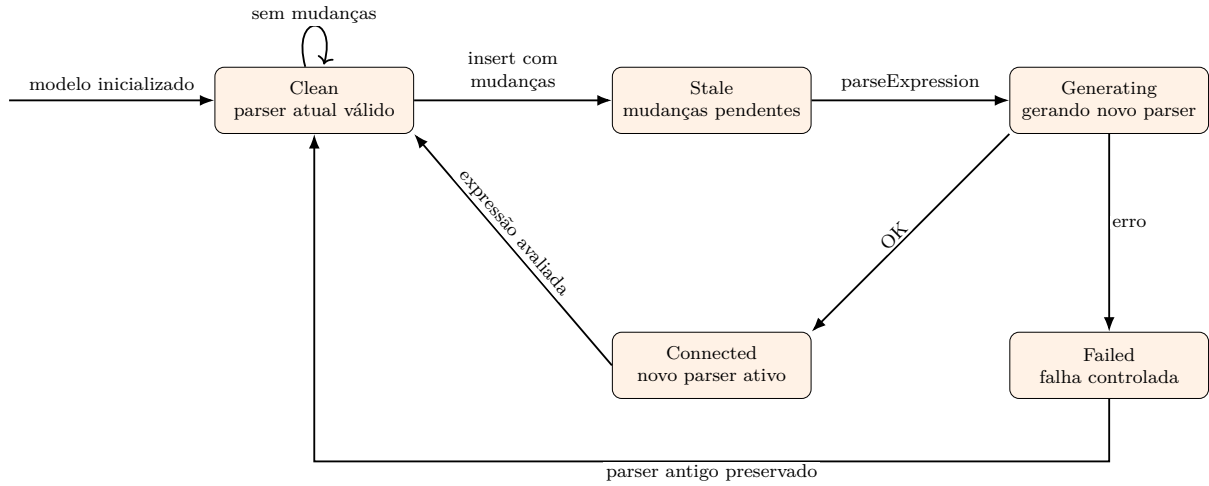


Figura 4: Estados do parser dinâmico e tratamento de falhas.

3 Validação

A validação foi organizada para demonstrar os requisitos funcionais do enunciado: declaração de mudanças por uma extensão, agregação das mudanças, geração e compilação do parser, conexão do novo parser, reconhecimento de funções novas e comportamento controlado em falhas.

3.1 Rastreabilidade entre requisitos, contratos e testes

A Tabela 2 explicita como os casos de uso do mecanismo foram validados. O foco está nos requisitos do parser dinâmico e nas mudanças desta branch; falhas antigas de testes não relacionados ao parser não foram consideradas como evidência contra este componente.

Tabela 2: Rastreabilidade dos requisitos funcionais do parser dinâmico.

Requisito ou caso de uso	Contrato relacionado	Implementação validada	Teste principal
Extensão declara alterações de linguagem.	<code>ModelDataDefinition::getParserChangesInformation()</code> e <code>ParserChangesInformation</code> .	Objeto de mudanças armazenado em <code>includes</code> , <code>tokens</code> , <code>produções</code> e <code>regras léxicas</code> ; <code>hasChanges()</code> distingue contribuição vazia de alteração real.	<code>ParserChangesInformation*</code>
Parser coleta mudanças de uma extensão carregada no modelo.	<code>ParserManager::aggregateChanges()</code> .	Varredura das definições de dados do modelo, descarte de contribuições nulas ou vazias e retorno das alterações efetivas.	<code>DynamicParserExtensionAddsNewFunction</code>

Requisito ou caso de uso	Contrato relacionado	Implementação dada	validação	Teste principal
Múltiplas extensões podem contribuir simultaneamente.	Lista de <code>ParserChangesInformation</code> agregada pelo <code>ParserManager</code> .	Combinação dos fragmentos de tokens, includes, produções e regras léxicas antes da geração.		<code>AggregateChangesCombinerMultipleParserExtensions</code>
Novo parser é gerado a partir dos arquivos-base.	<code>ParserManager::generateNewParser()</code> .	Injeção nos marcadores Bison/Flex, criação de arquivos no diretório de trabalho, execução de Bison/Flex e compilação da <code>.so</code> .		<code>DynamicParserExtensionAddsNewFunction</code>
Parser gerado é conectado ao modelo.	<code>ParserManager::connectNewParser()</code> e <code>Parser_if</code> .	Carregamento dinâmico da biblioteca, resolução da <code>factory genesys_createParser</code> , criação de <code>ParserDefaultImpl2</code> e substituição do parser ativo.		<code>DynamicParserExtensionAddsNewFunction</code>
Atualização ocorre automaticamente, sem intervenção manual.	<code>ModelDataManager::insert()</code> e <code>Model::parseExpression()</code> .	Inserção marca o modelo como potencialmente desatualizado; a próxima chamada de <code>parseExpression()</code> executa a regeneração antes de avaliar a expressão.		<code>LazyDynamicParserReloadsOnNextParseExpression</code>
Falhas de geração ou conexão não inutilizam o simulador.	Contrato de substituição segura do parser.	Erro é retornado ao chamador, a conexão falha de forma controlada e o parser anterior permanece capaz de avaliar expressões já suportadas.		<code>LazyDynamicParserFailurePreservesExistingParser</code>
Sampler continua com ownership correto após troca de parser.	<code>Parser_if::releaseSampler()</code> e <code>Parser_if::setSamplerOwned()</code> .	Transferência explícita do sampler do parser antigo para o parser novo, evitando dupla liberação e vazamento.		<code>ParserDefaultImpl2SetSampler*</code>

3.2 Testes unitários e de integração

Teste	Evidência fornecida	Resultado
<code>ParserChangesInformationStartsEmpty</code>	Valida que o contrato de mudanças inicia sem fragmentos configurados.	Passou
<code>ParserChangesInformationStoresAllConfiguredSections</code>	Valida armazenamento dos campos de mudanças, incluindo tokens, includes e produções.	Passou

Teste	Evidência fornecida	Resultado
<code>ParserChangesInformationReportsWhetherAnySectionHasChanges</code>	Valida a detecção de uma mudança efetiva no parser.	Passou
<code>DynamicParserExtensionAddsNewFunction</code>	Executa o pipeline end-to-end: extensão declara mudanças, parser é gerado, compilado, conectado e passa a reconhecer <code>parsertest</code> .	Passou
<code>LazyDynamicParserReloadsOnNextParseExpression</code>	Demonstra atualização automática sob demanda: a primeira chamada a <code>parseExpression</code> recompila e conecta o parser.	Passou
<code>AggregateChangesCombinesMultipleParserExtensions</code>	Valida agregação de mudanças de múltiplas extensões e reconhecimento de <code>firsttest</code> e <code>secondtest</code> .	Passou
<code>LazyDynamicParserFailurePreservesExistingParser</code>	Injeta uma mudança inválida e comprova falha controlada, com preservação do parser antigo.	Passou
<code>ConnectNewParserRejectsMissingLibraryWithMessage</code>	Valida diagnóstico para falha de conexão quando a biblioteca dinâmica não existe.	Passou
<code>ParserDefaultImpl2SetSampler*</code>	Valida ownership do <code>Sampler_if</code> durante troca de parser, evitando delegação indevida.	Passou

3.3 Comandos de validação

Os testes focados no escopo do parser dinâmico foram executados com os seguintes comandos:

```
./build/tests-kernel-unit/source/tests/unit/genesys_test_parser_expressions \
  --gtest_filter='ParserExpressionsTest.DynamicParserExtensionAddsNewFunction:\
ParserExpressionsTest.LazyDynamicParserReloadsOnNextParseExpression:\
ParserExpressionsTest.AggregateChangesCombinesMultipleParserExtensions:\
ParserExpressionsTest.LazyDynamicParserFailurePreservesExistingParser:\
ParserExpressionsTest.ConnectNewParserRejectsMissingLibraryWithMessage:\
ParserExpressionsTest.ParserDefaultImpl2SetSampler*'
```

```
./build/tests-kernel-unit/source/tests/unit/genesys_test_simulator_support \
  --gtest_filter='SimulatorSupportTest.ParserChangesInformation*'
```

```
./build/tests-kernel-unit/source/tests/unit/genesys_test_support_parsermanager
```

Os resultados observados foram: 7/7 testes do fluxo dinâmico do parser aprovados, 4/4 testes de `ParserChangesInformation` aprovados e 3/3 testes de suporte de `ParserManager` aprovados. Testes antigos fora do escopo do mecanismo de parser dinâmico não foram usados como critério de validação deste componente.

3.4 Experimento com plugin Demo

Além dos testes automatizados, foi realizado um experimento visual com um plugin de demonstração, usado durante o desenvolvimento para comprovar o comportamento observável do mecanismo de parser dinâmico na interface do GenESyS. Esse plugin não compõe o escopo final da branch principal do trabalho; ele funciona como evidência experimental e didática de que uma extensão concreta consegue ensinar uma nova função à linguagem de expressões.

O experimento foi organizado em três momentos. Primeiro, avaliou-se a função `demo()` no console de expressões sem que a extensão estivesse presente. Como esperado, o parser base não reconheceu a função, pois ela não fazia parte da gramática estática original. Em seguida, a extensão Demo foi inserida no modelo, declarando as alterações necessárias para o lexer e para a gramática. Por fim, a expressão com `demo()` foi executada novamente, agora com o parser atualizado.

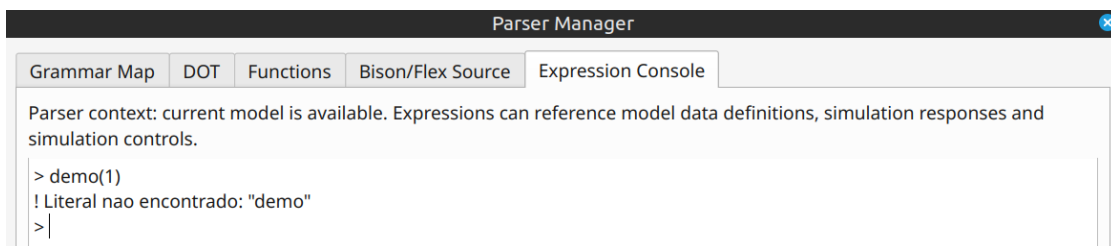


Figura 5: Antes da extensão Demo: o console de expressões não consegue avaliar `demo()`, pois a função ainda não pertence à linguagem reconhecida pelo parser.

O comportamento anterior confirma a condição inicial do experimento: a linguagem base não conhece a função `demo`. A contribuição da extensão Demo era intencionalmente simples, para isolar o mecanismo arquitetural: reconhecer a palavra `demo`, aceitar a forma sintática `demo(expressao)` e calcular o valor da expressão interna acrescido de uma unidade.

Listing 1: Fragmento experimental do DemoPlugin usado para declarar a função `demo(x)`.

```
ParserChangesInformation* DemoPlugin::_getParserChangesInformation() {
    ParserChangesInformation* changes = new ParserChangesInformation();
    changes->setTokens("%token <obj_t> fDEMO\n");
    changes->setFunctionProductions(
        "    | fDEMO \"(\" expression \")\" { $$.valor = $3.valor + 1; }\n"
    );
    changes->setLexicalRules(
        "[dD] [eE] [mM] [oO] "
        "{return yy::genesyspp_parser::make_fDEMO("
        "obj_t(0, std::string(yytext)), loc);}\n"
    );
    return changes;
}
```

O trecho mostra as três partes essenciais do contrato. A declaração de token informa ao Bison que existe um novo símbolo terminal, chamado `fDEMO`. A produção gramatical define que esse token pode aparecer como chamada de função recebendo uma expressão entre parênteses. A regra léxica informa ao Flex que as variações de capitalização da palavra `demo` devem produzir o token `fDEMO`. A ação semântica da produção implementa o comportamento experimental $\text{demo}(x) = x + 1$.

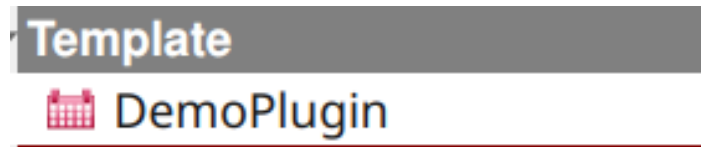


Figura 6: Inserção da extensão Demo no modelo, tornando disponível uma contribuição de parser para a próxima atualização dinâmica.

Após a inserção, o mecanismo implementado passa a ter informação suficiente para regenerar o parser sob demanda. A extensão não edita diretamente os arquivos `.yy` e `.ll`; ela apenas devolve um objeto `ParserChangesInformation`. O `ParserManager` permanece responsável por agregar as mudanças, injetar os fragmentos nos marcadores corretos, executar Bison/Flex, compilar a biblioteca dinâmica e conectar a nova instância do parser ao modelo.

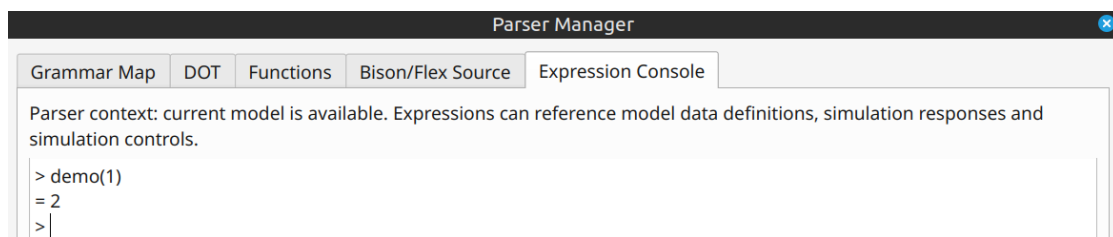


Figura 7: Depois da atualização dinâmica: a função `demo()` passa a ser reconhecida e executada pelo console de expressões.

O resultado experimental confirma o mesmo comportamento observado nos testes automatizados, mas por uma perspectiva de uso da ferramenta. A falha inicial demonstra que a função não existia na gramática base; a inserção da extensão demonstra o ponto de entrada do contrato; e a execução final demonstra que o parser ativo foi atualizado com sucesso para reconhecer uma construção nova. Como observação final, o código do plugin Demo não permaneceu na versão final do projeto, pois representava apenas um experimento visual de validação. Os testes automatizados descritos anteriormente indicam de forma mais direta os resultados e a eficácia da implementação.

4 Conclusão

O componente desenvolvido transforma a infraestrutura de extensão do parser do GenESyS em um mecanismo funcional. Plugins e extensões podem declarar mudanças por meio de `ParserChangesInformation`; o `ParserManager` coleta essas mudanças, injeta os fragmentos nos arquivos-base do lexer e da gramática, regenera o parser com Flex/Bison, compila uma biblioteca dinâmica e conecta a nova instância ao modelo.

A decisão de usar atualização automática sob demanda atende ao objetivo de adaptação em tempo de execução sem impor recompilações desnecessárias. A validação por testes demonstra tanto o caso de sucesso quanto situações de falha, incluindo preservação do parser antigo e ownership seguro do sampler.

Assim, o trabalho atende ao objetivo central do tema: permitir que a linguagem de expressões do GenESyS deixe de ser uma gramática fixa e passe a refletir, em tempo de execução, as extensões carregadas no simulador.

Referências

- [1] Free Software Foundation. *GNU Bison Manual*. Disponível em: <https://www.gnu.org/software/bison/manual/>.
- [2] The Flex Project. *Flex Manual*. Disponível em: <https://westes.github.io/flex/manual/>.
- [3] Kitware. *CMake Documentation*. Disponível em: <https://cmake.org/documentation/>.
- [4] R. L. Cancian. *GenESyS Simulator: código-fonte e arquitetura do simulador*. Repositório do projeto utilizado como base do desenvolvimento.